

1.Tool Contract란 무엇인가

Tool Contract란 무엇인가

Tool Contract는 Agent와 Tool 사이의 사용 규칙입니다. 여기서 Tool은 개발자가 사용하는 일반적인 도구가 아니라, Agent가 업무를 수행하기 위해 호출할 수 있는 기능 단위입니다. 예를 들어 설비 상태 조회, 알람 이력 확인, 공정 상태 확인, 품질 지표 조회, 정비 이력 확인, 기술 문서 검색 같은 기능을 Agent Tool로 볼 수 있습니다.

Tool Contract는 이러한 기능을 Agent가 언제, 어떤 입력으로, 어떤 결과를 기대하며 호출해야 하는지 정리하는 기준입니다. 특정 JSON 파일 하나만을 의미하는 것이 아니라, Agent 운영 품질을 좌우하는 설계 문서로 이해해야 합니다.

Tool Contract 구성요소

Tool Contract는 단순한 함수 명세가 아니라, Agent가 Tool을 올바르게 선택하고 호출하기 위한 설계 규칙 전체를 포함합니다. 각 구성요소가 어떤 역할을 하는지 설계 관점에서 살펴봅니다.

name — Tool 식별 이름

Agent와 LLM이 Tool을 구분하는 이름입니다. 단순 식별자가 아니라 업무 목적을 드러내야 합니다. 예: `get_recent_alarm_events`, `search_manual`

description — Tool 선택 기준

언제 이 Tool을 사용할지 알려주는 선택 기준입니다. LLM이 여러 Tool 중 적절한 기능을 고르는 데 직접 영향을 줍니다.

input_schema / output_schema

어떤 입력을 받을 수 있는지 제한하는 구조(input)와, Agent가 결과를 일관되게 해석할 수 있게 만드는 반환 구조(output)입니다.

error_schema — 오류 처리 기준

실패 상황을 숨기지 않고 해석 가능하게 남기는 기준입니다. DB 연결 실패, 조회 결과 없음, 입력값 누락, 검색 결과 부족은 서로 다른 의미를 가지므로 Agent가 구분해서 처리할 수 있어야 합니다.

example / validation rule

실제 호출 검증과 4일차 평가 기준으로 연결됩니다. 현업 적용 시에는 권한 조건, row limit, 반환 필드 제한, 실행 기록 항목까지 함께 포함해야 합니다.

① 현재 실습에서는 완성된 JSON schema 파일을 전제로 하기보다, 실제 함수 입력·출력, Tool 이름, Tool 설명을 기준으로 Contract 관점을 확인합니다. 실습 파일: `manufacturing_mcp_client.py`의 `SUPPORTED_TOOLS`와 `TOOL_DESCRIPTIONS`, `manufacturing_mcp_server.py`의 `@mcp.tool` 등록 구조, `postgres_db_tool.py`와 `search_manual_tool.py`의 함수 입력·출력 구조를 Tool Contract 관점으로 확인합니다.

Tool Description이 중요한 이유

LLM은 Tool 설명을 보고 어떤 Tool을 사용할지 판단합니다. 따라서 **description**은 단순 주석이 아니라 Tool 선택 기준입니다. 설명이 모호하면 알람 이력 조회가 필요한 상황에서 문서 검색 Tool을 선택하거나, 반대로 장애 대응 가이드 근거가 필요한 상황에서 DB Tool만 호출하는 문제가 발생합니다.

특히 제조 Agent처럼 설비 상태 조회, 알람 이력 조회, 품질 지표 조회, 기술 문서 검색이 함께 존재하는 구조에서는 Tool 설명의 품질이 전체 응답 품질에 직접 영향을 줍니다. 각 설명은 "무엇을 하는 Tool인지"뿐 아니라 "언제 사용해야 하는지"와 "무엇을 하지 않는지"까지 포함해야 합니다.

search_manual

기술 문서와 장애 대응 가이드를 검색하는 Tool입니다.
→ 실제 알람 발생 이력을 조회하는 Tool이 아닙니다.

get_recent_alarm_events

최근 알람 발생과 반복 여부를 확인하는 Tool입니다.
→ 원인 분석 보고서를 작성하는 Tool이 아닙니다.

manufacturing_mcp_client.py의 TOOL_DESCRIPTIONS는 이러한 관점을 확인하기 위한 실습 지점입니다. 이런 구분이 명확해야 LangGraph Routing과 Multi-Agent State 전달도 안정적으로 설계할 수 있습니다.

좋은 Description과 나쁜 Description

description의 품질은 LLM이 Tool을 올바르게 선택하는 데 결정적인 영향을 미칩니다. 나쁜 description은 기능을 너무 넓게 표현하고, 좋은 description은 사용 조건과 반환 목적을 구체화합니다. 아래 비교를 통해 설계 기준을 확인합니다.

Tool	나쁜 Description	좋은 Description
알람 이력 조회	"알람을 조회한다" — 어떤 입력을 기준으로 무엇을 확인하는지 불명확	equipment_id와 alarm_code 기준으로 최근 알람 발생 이력, 반복 여부, 마지막 발생 시각을 확인할 때 사용. 원인 분석 자체는 수행하지 않음
문서 검색	"문서를 검색한다" — 어떤 문서를, 어떤 목적으로 검색하는지 불명확	알람 코드, 증상, 설비 유형과 관련된 기술 매뉴얼이나 장애 대응 문서를 검색할 때 사용. 실제 알람 발생 이력은 조회하지 않음

제조 현업의 반복 알람 분석 상황에서는 DB Tool이 실제 이력과 수치 근거를 제공하고, RAG Tool이 기술 문서와 조치 절차 근거를 제공합니다. description은 이 경계를 분명히 표현해야 합니다.

- ❑ 결국 좋은 description은 Tool의 기능 설명이 아니라, LLM이 Tool을 선택할 때 참고하는 운영 규칙입니다. Tool 분할은 코드 문제가 아니라 업무 설계 문제입니다.

실습 예제: TOOL_DESCRIPTIONS와 MCP Tool 등록 확인

현재 Day3 예제에서는 별도 `mcp_tool_contracts.json` 파일을 직접 확인하는 방식이 아니라, `manufacturing_mcp_client.py`의 `SUPPORTED_TOOLS`와 `TOOL_DESCRIPTIONS`, `manufacturing_mcp_server.py`의 `@mcp.tool` 등록 구조를 확인합니다.

MCP는 Agent가 외부 기능을 호출할 때 사용하는 표준 연결 방식으로 이해할 수 있으며, 여기서는 제조 DB, 설비 로그, 기술 문서 검색 같은 기능을 Agent가 일관된 규격으로 호출하게 만드는 구조로 다룹니다.



이 장의 핵심은 실행 성공보다 Tool 설명과 등록 구조를 **Contract** 관점으로 읽는 것입니다.

선택 실행 명령 (Windows PowerShell):

```
uv run python  
src/day3/manufacturing_mcp_client.py
```

Tool 이름이 업무 목적을 드러내는 지 확인

`get_recent_alarm_events`,
`get_equipment_status`, `search_manual`
등 이름만 보고도 어떤 업무 기능인지 파악할 수 있어야 합니다.

description이 언제 사용할지 설명하는지 확인

각 Tool의 description이 "무엇을 하는지"뿐 아니라 "언제 사용해야 하는지", "무엇을 하지 않는지"까지 포함하는지 검토합니다.

입력값과 반환 구조가 Agent가 읽기 쉬운지 검토

알람 이력 확인 Tool은 `equipment_id`와 `alarm_code` 같은 입력이 명확해야 하고, 기술 문서 검색 Tool은 `alarm_code`, `symptom`, `user_query`, `top_k`처럼 검색 목적을 표현할 수 있어야 합니다.

Text-to-SQL vs 정해진 Query 방식

Agent가 DB에 접근하는 방식은 크게 두 가지로 나눌 수 있습니다. 설계 관점에서 각 방식의 장단점과 제조 현업에서의 적합성을 비교합니다.

Text-to-SQL 방식

자연어 질문을 SQL로 변환할 수 있어 유연합니다. 사용자가 "최근 반복 알람이 많은 설비를 찾아줘"라고 질문하면 SQL을 동적으로 생성해 다양한 조회를 시도할 수 있습니다.

제조 현업에서의 위험:

- 잘못된 테이블에 접근하거나 과도한 범위 조회
- 권한 범위를 넘어서는 데이터 반환
- 민감정보 노출 가능성
- SQL 생성 결과를 항상 신뢰하기 어려움

정해진 Query를 Tool로 감싸는 방식

유연성은 낮지만 PoC 초기와 운영 검증 단계에서 안정적입니다. 현재 `postgres_db_tool.py`는 정해진 조회 함수를 Tool처럼 사용하는 구조로 볼 수 있습니다.

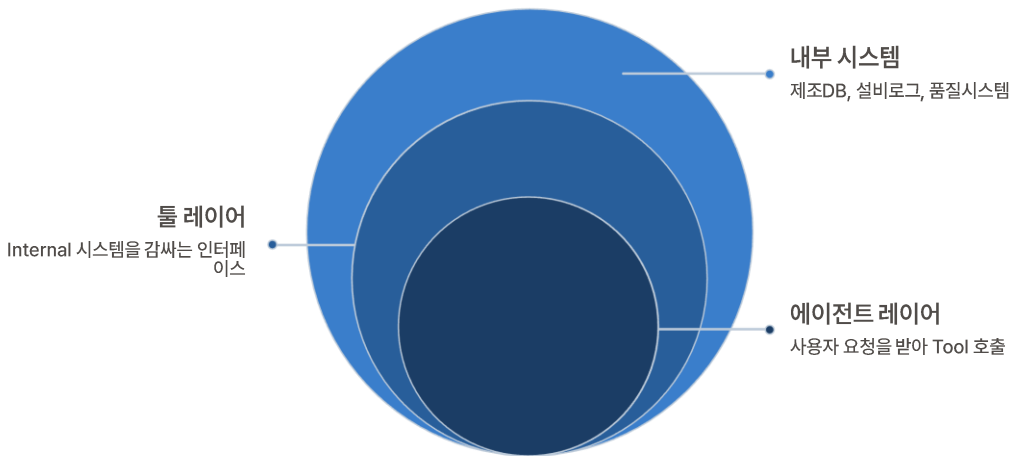
운영 단계에서 필요한 기준:

- read-only 권한
- row limit 설정
- allowlist 기반 조회
- query validation
- 감사 로그 기준

설비 상태 조회, 알람 이력 조회, 품질 지표 조회처럼 검증된 Query를 함수로 감싸면 입력값과 반환 범위를 통제하기 쉽습니다. 이 관점은 다음 장의 DB/Log Tool 설계로 이어지며, DB 기능을 Agent가 직접 다루지 않고 Tool 계층 뒤에 두는 이유를 설명합니다.

교육용 PostgreSQL의 역할

교육용 PostgreSQL은 실제 사내 DB가 아니라 내부 시스템 연동 구조를 재현하기 위한 샘플 환경입니다. 이 장에서 중요한 것은 PostgreSQL 자체를 배우는 것이 아니라, 제조 DB와 설비 로그 기능을 Agent Tool 뒤에 두는 구조를 이해하는 것입니다. 수업의 목적은 SQL 문법 학습이 아니라 제조 DB/Log 기능을 Agent가 호출 가능한 Tool로 감싸는 설계 관점을 익히는 데 있습니다.



현업에서는 PostgreSQL 대신 사내 제조 DB, 설비 로그 시스템, 품질 시스템, 정비 이력 시스템이 연결될 수 있습니다. Agent가 이러한 시스템에 직접 접근하도록 만들면 권한, 조회 범위, 오류 처리, 감사 로그 기준이 흩어질 수 있습니다.



DB는 "Agent가 직접 만지는 대상"이 아니라 "Tool 계층 뒤에 놓이는 내부 시스템"으로 이해해야 합니다. 예를 들어 설비 반복 알람 원인을 확인하는 Agent는 제조 DB를 직접 조회하는 것이 아니라, 설비 상태 조회 Tool과 최근 알람 이력 조회 Tool을 통해 필요한 근거만 받아야 합니다. 이러한 구조가 있어야 4일차의 실행 기록 평가와 Guardrail 검증으로 연결할 수 있습니다.

제조 DB/Log Tool 구조

postgres_db_tool.py는 제조 DB/Log 기능을 업무 목적별 함수로 나눈 예제입니다. 각 함수가 어떤 업무 근거를 제공하는지 설계 관점에서 확인합니다.

get_equipment_status

설비 기본 정보와 설비가 속한 라인 또는 공정 정보를 확인하는 기능입니다.

get_recent_alarm_events

특정 설비나 알람 유형에 대해 최근 알람 이력을 조회하고 반복 발생 여부를 확인합니다.

get_process_status

최근 공정 상태를 확인합니다. 온도, 압력 등 공정 파라미터 이상 여부 판단에 활용됩니다.

get_quality_metrics

품질 지표 변화나 수율, 불량률 같은 수치 근거를 확인하는 데 사용합니다.

get_maintenance_history

최근 정비 이력과 부품 교체 이력 확인에 연결됩니다.

get_equipment_overview

여러 정보를 한 번에 묶어 조회하는 통합 Overview Tool입니다. 한 번의 호출로 설비 상황을 넓게 확인할 수 있지만, 역할이 커지고 반환 데이터가 많아질 수 있습니다.

현업에서는 통합 Tool과 개별 Tool의 균형을 검토해야 합니다. 반복 알람 분석 Agent가 처음에는 Overview Tool로 기본 상황을 파악하고, 이후 알람 이력이나 품질 지표를 개별 Tool로 추가 확인하는 방식도 가능합니다. 중요한 것은 각 Tool이 어떤 업무 근거를 제공하는지 명확히 구분하는 것입니다.

DB Tool 입력·조회·반환 구조

DB Tool은 equipment_id, alarm_code, line_id, limit 같은 제한된 입력값을 사용합니다. 이러한 입력값은 단순 함수 파라미터가 아니라 조회 범위를 통제하는 설계 장치입니다.

입력값 설계 관점

SELECT 조건과 LIMIT도 단순 SQL 요소가 아니라 과도한 조회를 막고, Agent가 필요한 범위의 데이터만 받을 수 있게 만드는 기준입니다.

제조 현업에서는 설비 단위, 라인 단위, 기간, 조회 건수 같은 조건이 명확하지 않으면 불필요하게 많은 데이터가 반환되거나 권한 범위를 넘어서는 조회가 발생할 수 있습니다.

반환 구조 설계 관점

현재 실습 코드는 Agent가 읽기 쉬운 dict 또는 list 형태로 결과를 반환합니다. 현업 적용 시에는 Agent가 결과를 안정적으로 해석할 수 있도록 응답 구조와 오류 처리 기준을 설계해야 합니다.

조회 결과 없음, DB 연결 실패, 입력값 누락, 권한 부족은 서로 다른 의미를 가지므로 같은 빈 결과로 처리하면 안 됩니다.

 반환 데이터도 Agent에게 필요한 최소 필드만 전달하고, 민감정보나 과도한 상세 데이터는 제한해야 합니다. 오류 처리 기준이 명확하지 않으면 Agent가 잘못된 판단을 내릴 수 있습니다.

MCP Tool 연계 동작 점검 체크리스트

MCP Tool 연계에서는 실행 성공 여부만 확인하면 안 됩니다. 어떤 Tool이 어떤 입력으로 호출되었고, 반환값이 Agent가 해석할 수 있는 구조인지 함께 점검해야 합니다.

Tool 호출 오류

- MCP 서버가 실행되어 있는가?
- Tool 이름이 등록된 이름과 일치하는가?
- Agent가 존재하지 않는 Tool을 호출하지 않았는가?

입력값 오류

- equipment_id, alarm_code, line_id, top_k 같은 필수 입력이 누락되지 않았는가?
- 조회 범위가 과도하지 않은가?
- 입력값이 Tool의 목적과 맞는가?

응답 형식 오류

- 반환값이 Agent가 기대하는 dict 또는 list 구조인가?
- result_count, results, error, message 같은 필드가 일관되게 제공되는가?
- 빈 결과와 오류 결과가 구분되는가?

조회 결과 없음

- 데이터가 없는 상황을 정상 근거처럼 처리하지 않았는가?
- Agent가 “조회 결과 없음”을 명확히 표시하는가?

검색 결과 부족

- RAG 검색 결과가 부족한데 조치 절차를 확정적으로 제시하지 않았는가?
- 검색 결과 부족 상태가 rag_results 또는 agent_steps에 남는가?

실습 예제: postgres_db_tool.py를 설계 관점으로 읽기

postgres_db_tool.py는 SQL 코드로만 읽는 파일이 아니라, 내부 제조 DB 기능을 Agent Tool로 감싸는 계층으로 읽어야 합니다. 각 코드 요소가 어떤 설계 의미를 갖는지 확인합니다.

→ DB 연결 정보

현업에서는 보안 저장소, 네트워크 정책, 권한 관리와 연결됩니다. 교육용 환경에서는 .env 파일로 관리하지만, 실제 사내 환경에서는 별도 보안 정책이 필요합니다.

→ SELECT 조건과 LIMIT

과도한 조회 방지, 기간 제한, row limit과 연결됩니다. 설비 반복 알람 분석 상황에서 어떤 조건으로 조회 범위를 제한하는지 확인합니다.

→ 반환 필드 구조

Agent에게 필요한 최소 데이터만 전달하는 기준과 연결됩니다.
get_equipment_overview는 통합 조회의 장점과 과도한 반환 위험을 함께 보여주는 예제입니다.

예를 들어 설비 반복 알람 분석 상황에서 어떤 함수가 설비 상태를 확인하고, 어떤 함수가 알람 이력을 확인하며, 어떤 함수가 품질 지표와 정비 이력을 제공하는지 읽어내는 것이 중요합니다. 이 관점이 있어야 DB 기능을 현업 Agent Tool로 대체할 때 수정 지점을 판단할 수 있습니다.

① 선택 실행 명령 (Windows PowerShell):

```
uv run python src/day3/postgres_db_tool.py
```

단, .env와 PostgreSQL 접속 정보가 준비되어 있어야 정상 실행될 수 있습니다. 실행이 어렵더라도 실습의 핵심은 함수 구조와 입력·출력 설계 관점으로 해석하는 것입니다.

현업 적용 시 수정해야 할 지점

교육용 구조를 실제 현업 구조로 어떻게 대체할지 판단하는 것이 이 장의 핵심입니다. 현업 적용 시에는 DB 접속 정보, 테이블 구조, 조회 권한, 반환 필드, row limit, 감사 로그, 오류 처리 기준을 수정해야 합니다.

1

권한 및 접근 범위

설비 담당, 공정 담당, 품질 담당이 접근할 수 있는 데이터 범위가 다를 수 있습니다. 특정 품질 지표나 정비 이력은 제한된 권한에서만 조회되어야 할 수 있습니다.

2

read-only 조회 Tool과 action Tool 분리

제조 DB Tool은 실제 조치 실행을 하지 않고, 근거 확인과 판단 자료 제공에 집중해야 합니다. 생산 조건 변경이나 작업 지시처럼 실제 업무 행동을 유발하는 기능은 별도 승인 절차와 Human Review를 거치도록 설계해야 합니다.

3

감사 로그 및 오류 기록

DB Tool의 반환 데이터는 Agent가 판단에 필요한 최소 정보로 제한하고, 조회 실패나 권한 부족 같은 오류는 실행 기록에 남겨야 합니다. 이 기록이 4일차 평가와 Guardrail 검증의 기반이 됩니다.

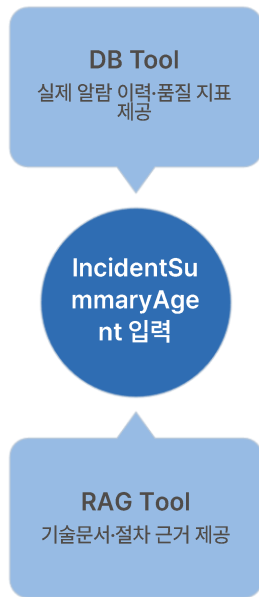


이 관점은 다음 장의 RAG Tool 구조와 연결됩니다. DB Tool은 실제 이력과 수치 근거를 제공하고, RAG Tool은 기술 문서와 조치 절차의 문서 근거를 제공합니다. 두 Tool의 역할 경계가 명확해야 Multi-Agent 흐름에서 근거를 구분할 수 있습니다.

RAG 검색도 Agent 입장에서는 Tool이다

2일차에는 RAG 검색 자체를 만들었다면, 3일차에는 그 RAG 기능을 Agent가 호출 가능한 Tool로 감쌉니다. search_manual Tool은 문서 검색 기능을 Agent가 사용할 수 있는 표준 업무 기능으로 만든 구조입니다.

DB 조회와 문서 검색을 동일한 Tool 호출 구조로 연결하면 Multi-Agent 흐름에서 함께 사용할 수 있습니다. 예를 들어 반복 알람 분석 Agent는 DB Tool로 실제 알람 이력과 품질 지표를 확인하고, search_manual Tool로 기술 문서와 장애 대응 가이드 근거를 검색할 수 있습니다.



RAG 검색 로직을 Agent 내부에 직접 섞지 않고 Tool 경계로 분리해야 합니다. 이렇게 분리하면 문서 저장소나 검색 방식이 바뀌어도 Agent의 상위 흐름을 비교적 안정적으로 유지할 수 있습니다. 현업에서 Markdown 문서가 사내 기술 문서 저장소로 바뀌거나, vector DB가 hybrid search로 바뀌더라도 search_manual이라는 Tool 경계가 유지되면 Agent는 동일한 방식으로 문서 근거를 요청할 수 있습니다.



이 구조가 DB 근거와 문서 근거를 구분하고, 이후 평가와 Guardrail 검증에서 어떤 근거가 사용되었는지 확인하는 기반이 됩니다.

search_manual_tool.py 구조

search_manual_tool.py는 기술 문서와 조치 절차를 검색하기 위한 RAG Tool입니다. 입력 구조와 처리 흐름, 출력 구조를 설계 관점에서 확인합니다.

입력 구조

alarm_code와 symptom은 검색 맥락을 좁히고, user_query는 사용자의 자연어 질문을 반영하며, top_k는 검색 결과 개수를 제한하는 기준이 됩니다.

처리 흐름은 검색 질의를 만들고, Day2 RAG search_top_k를 먼저 시도한 뒤, 실패하거나 결과가 없으면 CSV keyword search를 사용하는 fallback 구조입니다.

출력 구조

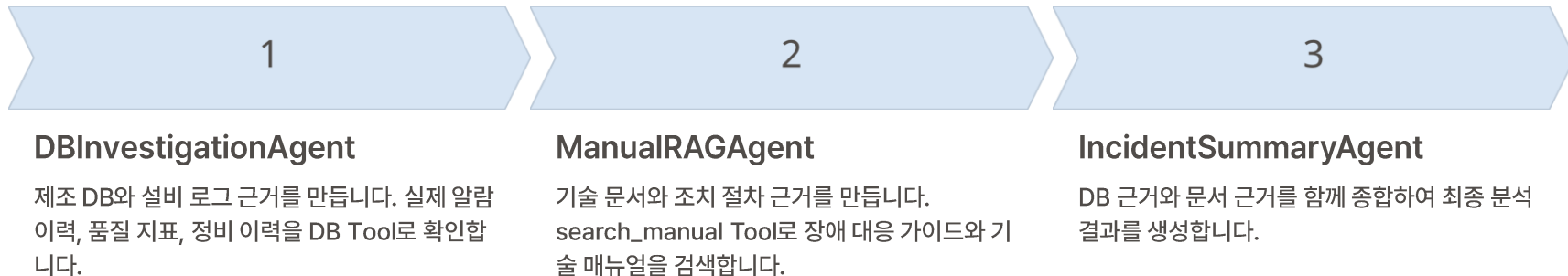
query, search_mode, top_k, result_count, results 구조로 이해할 수 있습니다.

search_mode는 어떤 검색 방식이 사용되었는지 확인하는 데 도움이 되고, result_count와 results는 Agent가 검색 결과의 충분성을 판단하는 근거가 됩니다.

 이 Tool은 실제 알람 발생 이력이나 설비 상태를 조회하지 않습니다. 실제 발생 데이터는 DB Tool이 담당하고, search_manual은 기술 문서, 장애 대응 가이드, 조치 절차 근거 검색을 담당합니다. 이 구분이 명확해야 Multi-Agent 흐름에서 DBInvestigationAgent와 ManualRAGAgent의 역할을 안정적으로 나눌 수 있습니다.

실습 예제: RAG를 Tool화하는 설계 의미

2일차에는 문서를 검색하는 RAG 자체를 만들었습니다. 3일차에는 그 RAG 기능을 Agent가 호출 가능한 search_manual Tool로 바꿉니다. search_manual은 DB Tool과 동일하게 Tool 이름, 입력, 출력, 오류 처리 기준을 가져야 합니다.



중요한 것은 search_manual이 Agent 내부에 섞인 검색 코드가 아니라, 독립된 Tool 경계로 분리되어 있다는 점입니다. 이 경계가 있어야 현업에서 문서 저장소, chunk 전략, metadata 구조, 검색 엔진을 바꾸더라도 Agent의 상위 흐름을 유지할 수 있습니다.

① 선택 실행 명령 (Windows PowerShell):
`uv run python src/day3/search_manual_tool.py`
단, Day2 RAG 또는 CSV metadata 준비 상태에 따라 검색 결과가 달라질 수 있습니다. 검색 결과가 비어 있더라도 구조 이해가 핵심입니다.

현업 적용 시 RAG Tool 수정 포인트

현업 적용 시에는 사내 문서 저장소, 문서 전처리 방식, chunk 전략, metadata 설계, vector DB 또는 hybrid search를 실제 환경에 맞게 바꿔야 합니다. search_manual Tool의 현업 전환은 검색 엔진 교체가 아니라 문서 접근, 근거 품질, 보안, 실행 기록 기준을 함께 설계하는 작업입니다.

문서 저장소 및 검색 방식

사내 기술 문서 저장소, chunk 전략, metadata 설계, vector DB 또는 hybrid search를 실제 환경에 맞게 교체합니다. 어떤 문서를 검색 대상으로 삼을지, 문서의 버전과 출처를 어떻게 남길지, 검색 결과의 관련성을 어떻게 평가할지까지 함께 검토해야 합니다.

보안 등급과 문서 접근 권한

설비 담당자가 볼 수 있는 문서와 품질 담당자가 볼 수 있는 문서가 다를 수 있으므로, RAG Tool도 권한 기준을 고려해야 합니다. 반복 알람 분석 상황에서 장애 대응 가이드는 검색할 수 있지만, 특정 제한 문서나 민감한 내부 절차 문서는 권한에 따라 제외되어야 할 수 있습니다.

검색 품질 평가 기준

RAG Tool은 단순 검색 기능이 아니라 Agent가 근거 문서를 찾기 위한 업무 기능입니다. 4일차에는 검색 결과가 적절했는지, 근거가 충분했는지, Guardrail이 필요한 지점이 어디인지 평가할 수 있습니다.

- ✔ 4일차 평가 연결: State와 agent_steps, db_results, rag_results를 실행 기록 관점으로 확인하면, DB Tool과 RAG Tool이 각각 어떤 근거를 제공했는지, Guardrail이 필요한 지점이 어디인지 판단할 수 있습니다. 이 관점이 5일차 사내 PoC 전환 시 무엇을 바꿔야 하는지 결정하는 기준이 됩니다.